

3.2P: Answer Sheet

Recall task 2.2P *Counter Class* and answer the following questions.

1. How many *Counter* objects were created?

Two Counter objects are created:

`myCounters[0] → new Counter("Counter 1")`

`myCounters[1] → new Counter("Counter 2")`

`myCounters[2] → points to the same object as myCounters[1], no new object created.`

2. Variables declared without the **new** keyword are different to the objects created using **new**. In the **Main** function, what is the relationship between the variables initialized with and without the **new** keyword?

Variables created with **new** (`myCounters[0]`, `myCounters[1]`) are new objects on the heap. `myCounters[2]` is just assigned the reference of `myCounters[1]`, pointing to the same object.

So with **new** creates an object, without **new** just copies a reference to an existing object.

3. In the **Main** function, explain why the statement `myCounters[2].Reset()`; also changes the value of `myCounters[0]`.

In the Main function, the statement `myCounters[2].Reset()`; does not change the value of `myCounters[0]` because `myCounters[0]` and `myCounters[2]` refer to different Counter objects.

`myCounters[0]` points to a Counter object created with "Counter 1", while `myCounters[2]` is a reference to the same object as `myCounters[1]` ("Counter 2").

Therefore, calling `Reset()` on `myCounters[2]` only affects the Counter object assigned to `myCounters[1]` and `myCounters[2]`, and does not affect `myCounters[0]`.

4. The difference between *heap* and *stack* is that heap holds “*dynamically allocated memory*.” What does this mean? In your answer, focus on the size and lifetime of the allocations.

Heap holds dynamic memory, meaning the memory for objects is allocated at runtime, not at compile time.
The size can be large, and objects exist until garbage collected (no longer referenced).

5. Are objects allocated on the heap or on the stack? What about local variables?

Objects (like Counter) are allocated on the heap.

Local variables (like myCounters) are stored on the stack and hold references to heap objects.

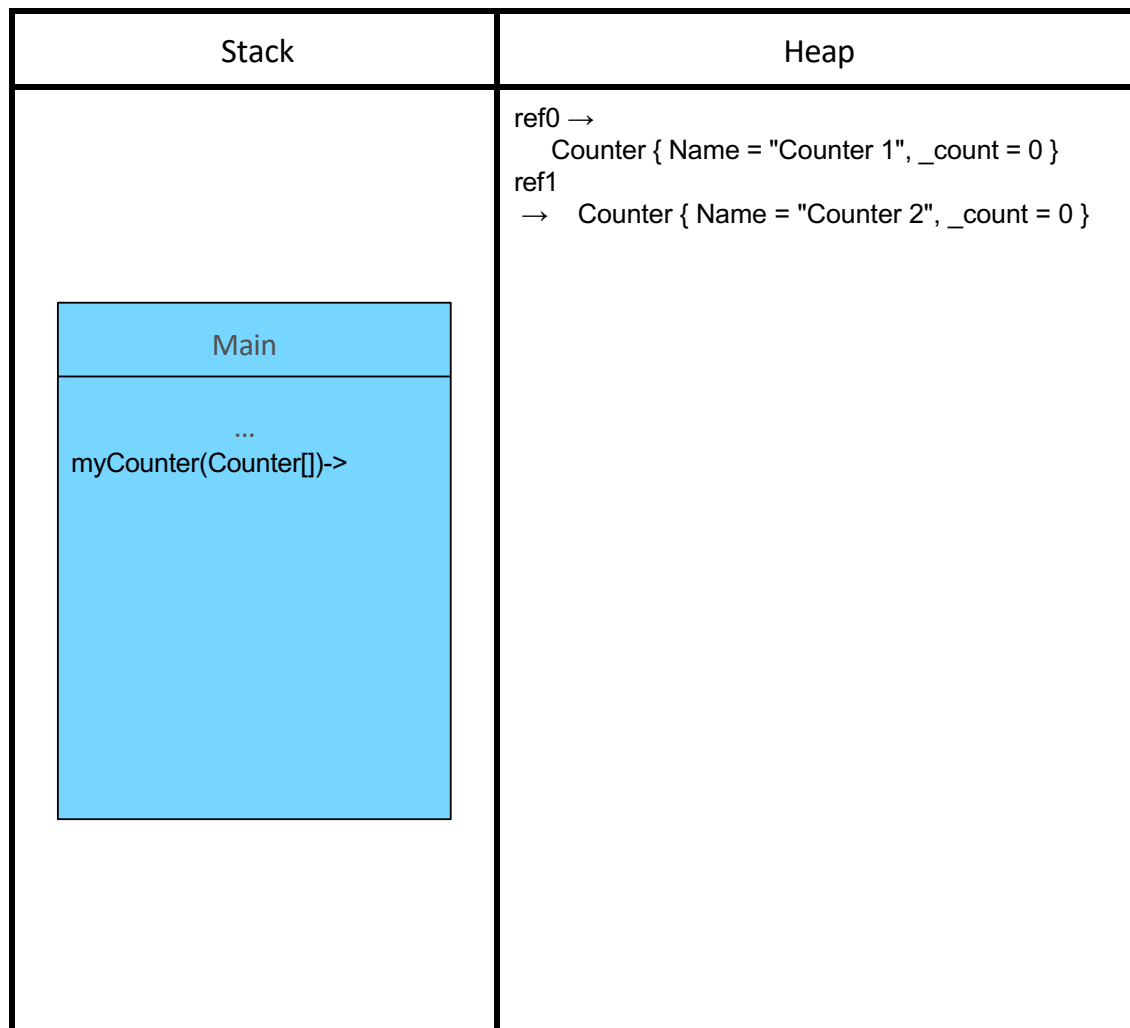
6. What is the meaning of the expression ***new*** *ClassName*(), where *ClassName* refers a class in your application? What is the value of this expression?

new *ClassName*() creates a new *ClassName* object on the heap.
The value returned is a reference (pointer) to that object.

7. Consider the statement “*Counter* ***myCounter***;”. What is the value of ***myCounter*** after this statement? Why?

After declaration, myCounter is null.
Because no object is created yet — it's just a reference without a target.

8. Based on the code you wrote in task 2.2P *Counter Class*, draw a diagram showing the locations of the variables and objects in function **Main** and their relationships to one another.



9. If the variable `myCounters` is assigned to null, then you want to change the value of `myCounters[X]`, where X is the last digit of your student ID, what will happen? Please provide your observation with screenshots and explanation.

```
CounterTask > Program.cs > Program > Main
1 namespace CounterTask
2 {
3     2 references
4     internal class Program
5     {
6         2 references
7         private static void PrintCounters(Counter[] counters)
8         {
9             foreach (Counter c in counters)
10             {
11                 Console.WriteLine("{0} is {1}", c.Name, c.Ticks);
12             }
13         }
14         0 references
15         public static void Main()
16         {
17             Counter[] myCounters = new Counter[3];
18             myCounters[0] = new Counter("Counter 1");
19             myCounters[1] = new Counter("Counter 2");
20             myCounters[2] = myCounters[1];
21
22             myCounters = null;
23             int X = 2;
24             myCounters[X] = new Counter("New Counter");
25
26             for (int i = 1; i <= 9; i++)
27             {
28                 myCounters[0].Increment();
29             }
30             for (int i = 1; i <= 14; i++)
31             {
32                 myCounters[1].Increment();
33             }
34         }
35     }
36 }
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
~/Desktop/OOPAssignments/P22/CounterTask main !40 ?31 > dotnet run
Unhandled exception. System.NullReferenceException: Object reference not set to an instance of an object.
at CounterTask.Program.Main() in /Users/khanthuaung/Desktop/OOPAssignments/P22/CounterTask/Program.cs:line 21
~/Desktop/OOPAssignments/P22/CounterTask main !40 ?31 > |
```

If myCounters is set to null, trying to access myCounters[X] will cause a NullReferenceException. This happens because myCounters no longer points to an array in memory. Since the reference is null, there is no object to access, and the program will crash when it tries to use it.

Hint. You may want to read this material for this task

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/null>

For further reading at your own.

- Null pointer CrowdStrike Bug, <https://www.thestack.technology/crowdstrike-null-pointer-blamed-rca/>
- CrowdStrike Blog, <https://www.crowdstrike.com/blog/tech-analysis-channel-file-may-contain-null-bytes/>

